

WEEK 3 (Day 1 & Day 2)

NumPy in Python

NumPy (Numerical Python) is a powerful Python library used for numerical computing. It provides support for multi-dimensional arrays, mathematical operations, and high-performance data processing. NumPy is widely used in Data Analytics, Data Science, Machine Learning, and Scientific Computing.

1. Creating a NumPy Array

```
1 import numpy as np
2 a = np.array([10,20])
3 b = np.array([40,50])
4 print(a)
5 print(b)
6 print(a*b)
7
```

[8]

[10 20]
[40 50]
[400 1000]

2. Checking the Type of an Array

```
import numpy as np
x = np.array([5,2,8,4,1])
print(type(x))
```

✓ [2] 300ms

<class 'numpy.ndarray'>

3. Creating a 2-Dimensional Array

```
1 y = np.array([[10,20,30,40],[50,50,60,70]])
2 print(y)
3
```

✓ [3] 170ms

[[10 20 30 40]
[50 50 60 70]]

4. Array Dimensions

```
> 1 y = np.array([[10,20,30,40],[50,50,60,70]])
  2 print(y)
  3 print("\nThe Dimensions of the given array are :",np.ndim(y))
✓ [6] 158ms

[[10 20 30 40]
 [50 50 60 70]]

The Dimensions of the given array are : 2
```

5. Array Shape

```
> 1 x = np.array([[1,2,3,6],[4,8,6,7],[14,1,2,47]])
  2 print(x)
  3 print("\nThe shape (rows & cols) of the given array is :",np.shape(x))
✓ [10] 134ms

[[ 1  2  3  6]
 [ 4  8  6  7]
 [14  1  2 47]]

The shape (rows & cols) of the given array is : (3, 4)
```

6. Array Size

```
> 1 x = np.array([[1,2,3,6],[4,8,6,7],[14,1,2,47]])      x
  2 print(x)
  3 print("\nThe shape (rows & cols) of the given array is :",np.shape(x))
  4 print("\nThe size of the array is :",np.size(x))
  5
✓ [11] 95ms

[[ 1  2  3  6]
 [ 4  8  6  7]
 [14  1  2 47]]

The shape (rows & cols) of the given array is : (3, 4)

The size of the array is : 12
```

7. Finding Maximum and Minimum

```
> 1 arr = np.array([12, 25, 8, 45])
  2
  3 print("Maximum:", arr.max())
  4 print("Minimum:", arr.min())
✓ [12] 110ms

Maximum: 45
Minimum: 8
```

8. Finding Mean and Sum

```
> 1 import numpy as np
  2
  3 arr = np.array([10, 20, 30, 40])
  4
  5 print("Sum:", arr.sum())
  6 print("Mean:", arr.mean()) arr
✓

Sum: 100
Mean: 25.0
```

9. Array Slicing

```
1 arr = np.array([10, 20, 30, 40, 50])
2
3 print(arr[1:4])
✓ [14] 57ms

[20 30 40]
```

10. Iterating Through an Array

```
> 1 import numpy as np
  2
  3 arr = np.array([10, 20, 30])
  4
  5 for i in arr:
  6     print(i)
✓ [15] 115ms

10
20
30
```

```
[18]: # concatenate

brr1 = np.array([12,4,6])
brr2 = np.array([10,52,14])
print(np.concatenate([brr1,brr2]))
```

```
[12  4  6 10 52 14]
```

```
[20]: arr1 = np.array([[10,20],[30,40]])
arr2 = np.array([[22,35],[65,48]])
print(np.hstack([arr1,arr2]))
```

```
[[10 20 22 35]
 [30 40 65 48]]
```

```
[21]: arr1 = np.array([[10,20],[30,40]])
arr2 = np.array([[22,35],[65,48]])
print(np.vstack([arr1,arr2]))
```

```
[[10 20]
 [30 40]
 [22 35]
 [65 48]]
```

```
•[26]: # splitting
a = np.array([10,20,30,40,60,90,80,70])
b = np.array_split(a,4)
print(b)
print(b[3])
```

```
[array([10, 20]), array([30, 40]), array([60, 90]), array([80, 70])]
[80 70]
```

```
[29]: # Adding and removing Elements in the array
a = np.array([10,20,50,44,690])
print(np.append(a,75))
print(np.insert(a,3,512))
print(np.delete(a,2))
```

```
[ 10  20  50  44 690  75]
[ 10  20  50 512  44 690]
[ 10  20  44 690]
```

```
[31]: # NumPy - Sort,Filter,Search
crr = np.array([[5,1,9,6,4,81],[7,4,8,5,6,14]])
print(np.sort(crr))
```

```
[[ 1  4  5  6  9 81]
 [ 4  5  6  7  8 14]]
```

```
[32]: arr1 = np.array([1,2,3,4,5,6,7,8,9])
a = np.where(arr1 % 2 == 0)
print(a)
```

```
(array([1, 3, 5, 7]),)
```

```
[34]: arr2 = np.array([20,40,60,80])
      fa = np.array([True,False,False,True]) # filter array(fa)
      new = arr2[fa]
      print(new)
```

```
[20 80]
```

```
[35]: arr2 = np.array([20,40,60,80])
      fa = arr2>45
      new = arr2[fa]
      print(new)
```

```
[60 80]
```

```
[37]: # Aggregating Functions
      b = np.array([5,10,15,20,25,30,35,40])
      print(np.size(b))
      print(np.max(b))
      print(np.min(b))
      print(np.sum(b))
      print(np.mean(b))
      print(np.cumsum(b))
      print(np.cumprod(b))
```

```
8
40
5
180
22.5
[ 5  15  30  50  75 105 140 180]
[  5  20  45  75 105 140 180 225 300 393750000 15750000000]
```

```
[46]: # Statistical Functions
      import statistics as stats
      a = np.array([100,250,140,865,125,745,562,250,148,500,250,460])
      print(np.mean(a)) # sum of all values/number of values
      print(np.median(a)) # central value after sorting
      print(stats.mode(a)) # frequently occurring value
      print(np.std(a)) # standard deviation (spread of data)
      print(np.var(a)) # variance
```

```
366.25
250.0
250
245.9089008149156
60471.1875
```

```
[47]: # coeff of correlation - if we are given a dataset and we have to find the relation
      # -1 represents inversely proportional relationship
      # 1 represents proportional relationship
      # 0 represents no relationship

      tobacco_conspt = np.array([10,30,50,30,45,10])
      deaths = np.array([100,120,112,100,110,120])

      print(np.corrcoef([tobacco_conspt,deaths]))

[[1.          0.05504085]
 [0.05504085 1.          ]]
```